

SunLock Lab

Solar Stewart Tracker

Technical Report

Realtime Embedded Systems — ENG5220 — University of Glasgow



Student ID	Name	Primary Contribution
3127931H	Fadi Halteh	System architecture, state machine, IMU/ADS1115 integration, diagrams
3137140W	Jichao Wang	3-RRS inverse kinematics, SystemFactory, AppConfig, entry structure
2429452Y	Ziming Yan	Vision subsystem (SunTracker), Qt GUI, live preview, charts
3139787A	Tareq A M Almzanin	Control layer, closed-loop logic, manual override, shared types
3099498Z	Zhenyu Zhu	PCA9685/ServoDriver, latency instrumentation, timing analysis

1. Introduction

The Solar Stewart Tracker is a realtime embedded system that continuously aims a solar-energy platform toward the sun. The mechanical platform is a 3-RRS Stewart-type parallel manipulator: a moving upper plate driven by three high-torque servos arranged at 120° intervals on a fixed base, connected through rod-end ball-joint linkages. A camera mounted above the platform captures live video. Software detects the sun position in each frame, calculates the error from centre, and drives the platform to correct it — all within a single C++17 Linux userspace process running on a Raspberry Pi 5.

The practical motivation is solar energy. A fixed panel only receives maximum irradiance at solar noon. Two-axis tracking increases daily energy yield by 25–40% at mid-latitudes. This project provides a low-cost embedded implementation using commodity hardware, open-source libraries, and a simple parallel linkage.

Beyond automatic tracking, the system supports two independently-driven manual modes: an operator can command the platform directly through potentiometers (read via an ADS1115 ADC with interrupt-driven acquisition) or through a Qt GUI. An MPU6050 IMU optionally provides platform-tilt feedback for improved accuracy.

The project is technically non-trivial because it combines four separate disciplines in one realtime pipeline: computer vision (brightness thresholding and weighted centroid estimation), proportional closed-loop control, 3-RRS Stewart-platform inverse kinematics (ZYX Euler rotation matrix, cosine-rule geometry per leg), and embedded hardware I/O (I2C, GPIO, PWM via PCA9685). All four must complete within one camera frame period — 33 ms at 30 fps.

1.1 Physical Behaviour

Input: IMX219 CSI camera at 30 fps. Output: PWM pulses on three servo channels via PCA9685 I2C driver board.

User interaction: headless CLI (manual/auto/set/threshold/quit/help commands) or Qt GUI with live camera preview, per-channel actuation charts, and manual position sliders. Potentiometer input uses interrupt-driven ADS1115 acquisition.

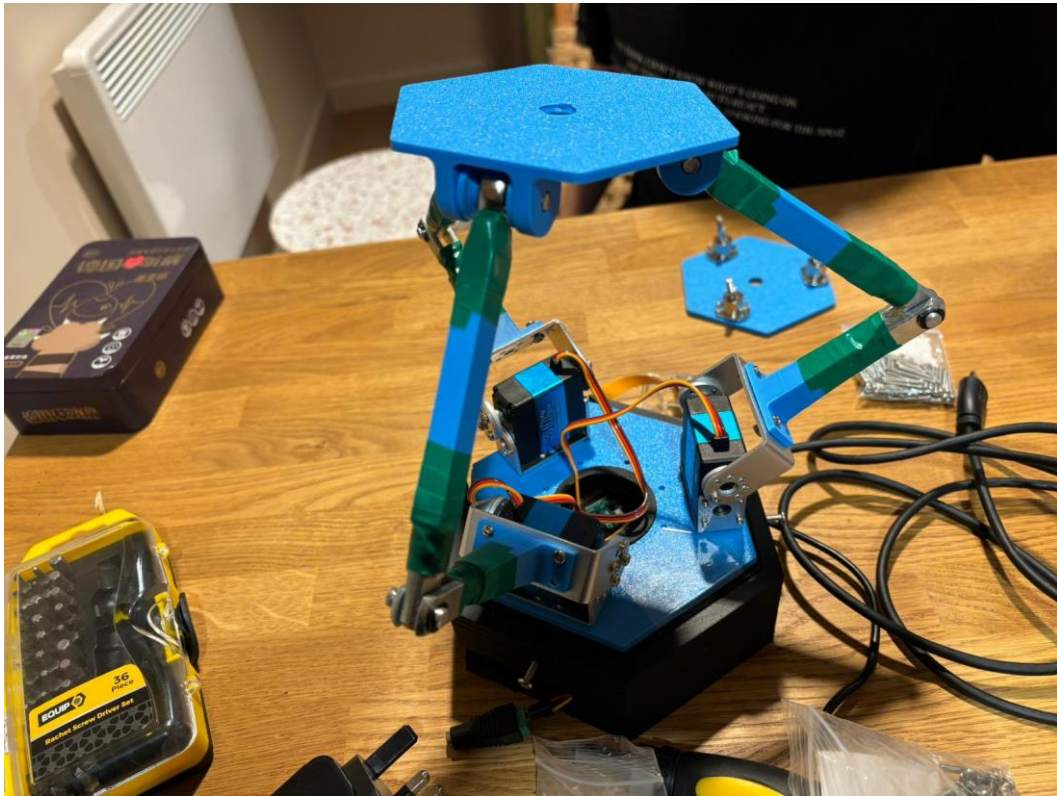


Figure 1.1. Full assembled Solar Stewart Tracker — 3-RRS platform, camera mount, and electronics enclosure.

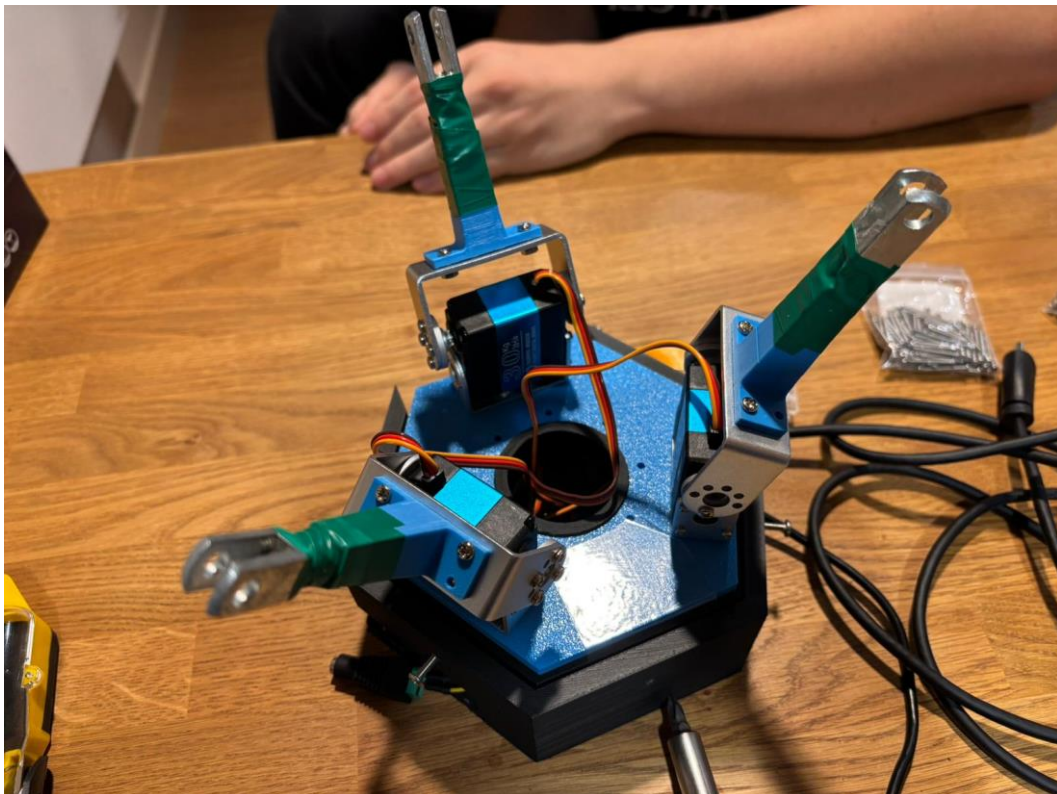


Figure 1.2. Section view showing servo arrangement, L1/L2 linkage arms, and upper plate.

2. System Overview

The entire system runs inside a single Linux userspace process. Hardware is accessed through standard kernel interfaces: I2C device files, the GPIO chip via libgpod, and the CSI camera via libcamera. There is no kernel module and no RTOS.

2.1 Sensing

Camera: an IMX219 8 MP module connects via CSI ribbon to the Raspberry Pi 5. Frames are delivered at 640×480, RGB888, 30 fps through a libcamera → libcamera2opencv bridge. Each completed DMA transfer fires a callback in the camera backend thread.

IMU: an MPU6050 accelerometer/gyroscope sits on I2C bus 1 at address 0x68. Its data-ready interrupt fires on GPIO27. The system reads 3-axis acceleration on each interrupt and uses the result for optional platform-tilt correction.

Manual input: an ADS1115 16-bit ADC sits on I2C bus 1 at address 0x48. Its ALERT/RDY pin fires on GPIO17 when a conversion is complete. Two channels read the wiper voltages from tilt and pan potentiometers.

2.2 Computation

The automatic pipeline runs entirely in a dedicated control thread: SunTracker finds the bright pixel cluster and emits a weighted centroid with confidence; Controller converts image-plane error into tilt/pan setpoints in radians; ManualImuCoordinator applies optional IMU correction; Kinematics3RRS solves the inverse kinematics to produce three servo angles in degrees; ActuatorManager applies clamping and rate limiting.

Manual paths are fully independent. Both the potentiometer path and the GUI path have their own threads and dispatch directly to Kinematics3RRS, without passing through the control thread and without any dependence on camera frames.

2.3 Actuation

ServoDriver converts degree-valued commands to PWM pulse widths and writes them to the PCA9685 PWM controller over I2C. The PCA9685 generates the actual PWM signals, which removes timing load from the processor. Three RDS3230 high-torque servos drive the 3-RRS linkage arms.

2.4 Operational States

State	Entry Condition	Behaviour
IDLE	Initial or after shutdown	System inactive. No frames acquired, no actuator output.
STARTUP	start() called	Camera null-check, servo driver start, queue reset, thread launch, sensor backend start.
NEUTRAL	STARTUP succeeds	Startup park position applied to all three servos.
SEARCHING	startup_mode == Auto	Full automatic pipeline active. Target confidence below threshold.
TRACKING	confidence ≥ threshold	Closed-loop automatic tracking. Sun centroid locked.
MANUAL	enterManual() from any state	Pot or GUI manual path active. Automatic pipeline suspended.
FAULT	Any hardware or kinematics failure	Actuator output suppressed. Requires explicit stop() to recover.
STOPPING	stop() from any state	Camera and backends stopped, threads joined, servos parked, state set to IDLE.

Table 2.1. Runtime state machine. All transitions are explicit and logged.

3. Hardware Requirements

This section covers every hardware component required to build and run the system in hardware mode. Table 3.1 lists the full bill of materials.

Component	Model / Specification	Interface	Role
Raspberry Pi 5 (8 GB)	BCM2712, Cortex-A76 quad-core, 64-bit	GPIO / CSI / I2C	Central processor. Runs the Linux userspace process.
Camera Module	IMX219 8 MP CSI Module	CSI ribbon (FPC)	30 fps frame source. Camera DMA completion wakes the control thread.
PCA9685 PWM Driver	16-ch I2C PWM, addr 0x40	I2C bus 1	Generates servo PWM signals independently of the processor.
Servo × 3	RDS3230, 20 kg·cm, 270° travel	PWM from PCA9685	Drives the three 3-RRS linkage arms.
MPU6050 IMU	3-axis accel/gyro, addr 0x68	I2C + GPIO27 INT	Platform-tilt feedback. INT pin fires when a sample is ready.
ADS1115 ADC	16-bit, 4-ch, addr 0x48	I2C + GPIO17 ALERT	Reads two potentiometers. ALERT/RDY fires when a conversion finishes.
Potentiometers × 2	10 kΩ linear rotary	Analog → ADS1115 A0/A1	Manual tilt and pan input.
External PSU	5–6 V, high-current	PCA9685 V+ rail	Servo power supply. Keeps the Pi from browning out under load.
3-RRS Platform	Acrylic / PLA 3D-printed	Mechanical	Parallel linkage with M6 rods and SA6TK ball joints.
FPC ribbon cable	Pi 5 CSI compatible	Camera interface	Connects IMX219 to the Pi CSI port.
Breadboard + jumpers	Standard prototyping kit	GPIO / I2C	I2C bus wiring and GPIO interrupt routing.
Fasteners	M2–M6 threaded rod, rod-ends, hex nuts	Mechanical	Platform arm assembly and length adjustment for calibration.

Table 3.1. Full bill of materials. Both ADS1115 and MPU6050 were chosen specifically because they provide hardware data-ready interrupt pins (ALERT/RDY and INT), which allow the software to block on a GPIO edge rather than polling I2C. Total estimated cost: £217.

3.1 I2C Bus and GPIO Assignments

Physical Pin	Signal	Direction	Connection
Pin 1	3.3 V	Power	PCA9685 VCC · ADS1115 VDD · MPU6050 VCC
Pin 3	GPIO2 SDA	Bidirectional	PCA9685 SDA · ADS1115 SDA · MPU6050 SDA
Pin 5	GPIO3 SCL	Bidirectional	PCA9685 SCL · ADS1115 SCL · MPU6050 SCL
Pin 6	GND	Power	All module grounds and external PSU ground (common ground)
Pin 11	GPIO17	Input	ADS1115 ALERT/RDY — conversion-complete interrupt via libgpiod

Physical Pin	Signal	Direction	Connection
Pin 13	GPIO27	Input	MPU6050 INT — data-ready interrupt via libgpiod
CSI	CSI lanes	Input	IMX219 camera module FPC ribbon

Table 3.2. Raspberry Pi 5 pin assignments. All three I2C devices share bus 1. GPIO17 and GPIO27 carry the hardware interrupt lines.

Critical wiring rules: all grounds must connect to a common plane; servo power must come from the external PSU through PCA9685 V+, never from Pi GPIO pins; PCA9685 has two separate power domains (VCC = 3.3 V logic; V+ = 5–6 V servo power); never swap SDA and SCL.

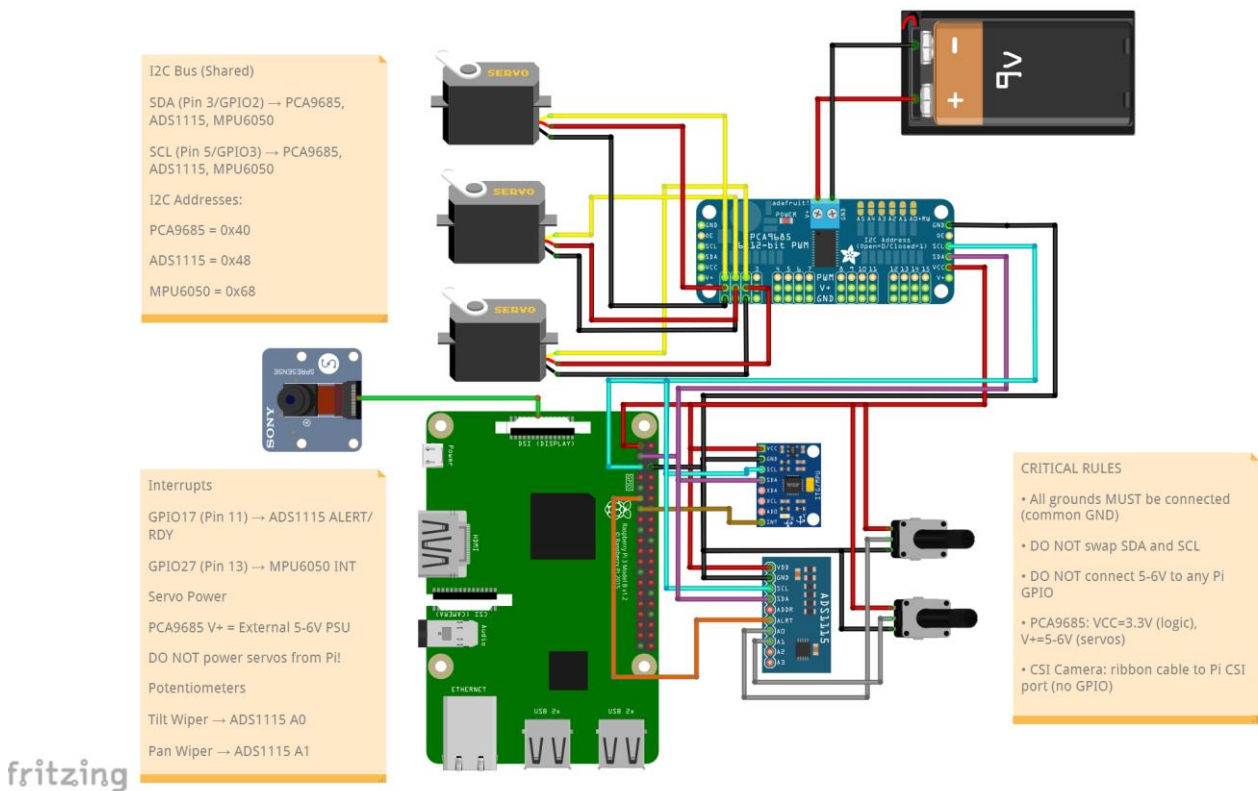


Figure 3.1. Circuit wiring diagram showing the shared I2C bus, GPIO17 and GPIO27 interrupt lines, servo power rail, and potentiometer inputs. Editable Fritzing source: [diagrams/circuit_diagram.fzz](#).

4. Mechanical Platform

The actuator mechanism is a 3-RRS Stewart-type parallel platform. The "3-RRS" label describes the three identical kinematic chains connecting the base to the moving plate, each with one Revolute joint (the servo horn), one Revolute joint (a ball-end clevis bracket), and one Spherical joint (a rod-end ball joint).

Three RDS3230 servos are fixed to the base plate at 120° intervals. Each servo drives an upper arm ($L1$ = horn length). The arm connects through a threaded M6 rod ($L2$ = rod length) with SA6TK rod-end ball joints to the moving upper platform. This arrangement gives two controlled degrees of freedom — pitch and roll — while yaw is constrained by the three-chain symmetry.

The inverse kinematics problem is: given a desired platform tilt expressed as pan and tilt in radians, compute the three servo horn angles needed to achieve that pose. Kinematics3RRS solves this using a ZYX intrinsic Euler rotation matrix, a cosine-rule calculation per leg, and branch selection that prefers horn-up configurations and continuity with the previous angle. If a configuration is geometrically unreachable, the solver falls back to the last valid command.

All geometric parameters — base radius (R_b), platform radius (R_p), home height (h), horn length ($L1$), rod length ($L2$), per-leg anchor angles — are calibration values in AppConfig. The full SolidWorks CAD assembly is in the repository at [diagrams/mechanical model/3rrs/](#).

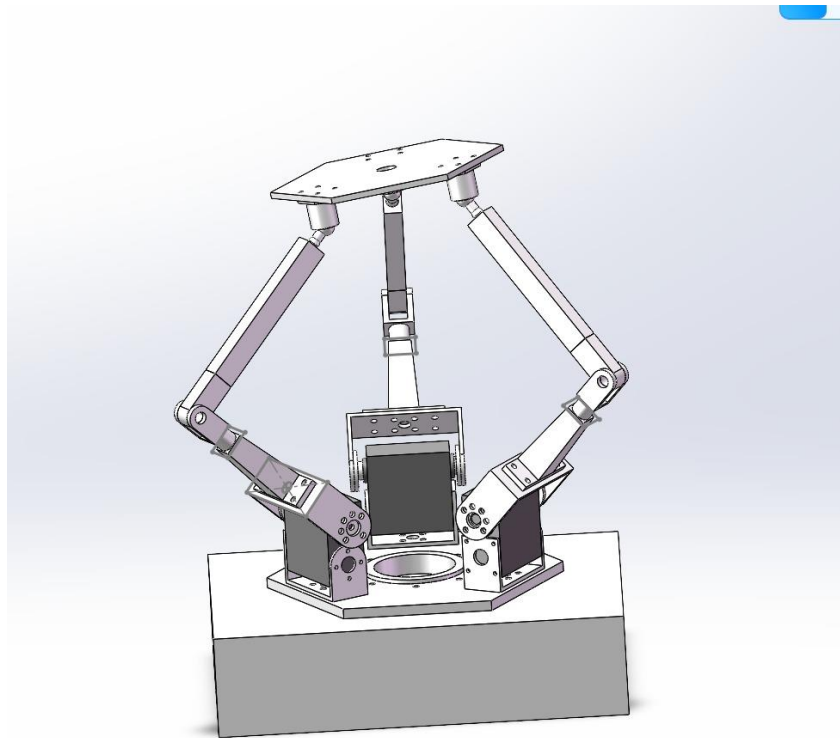


Figure 4.1. SolidWorks 3-RRS model showing base plate, three servo positions at 120° , $L1$ upper arms, M6 rod links with ball joints, and the moving upper platform.

5. Why Realtime?

The sun moves slowly roughly 2.7 pixels per second at 640×480 with a 60° horizontal field of view. The realtime requirement is not driven by the sun's angular speed. It is driven by something more fundamental: the control loop must respond to a physical event (a completed camera exposure) within a fixed deadline (the next frame period).

At 30 fps the deadline is 33 ms. Here is what happens if processing misses it:

- A frame arrives but the control thread is still processing the previous one. The new frame sits in the queue.
- When the control thread finishes, it picks up the queued frame — but that frame is already 33 ms old. The platform is now tracking a position from the past.
- Under sustained overload, queued frames accumulate. Each one adds 33 ms of lag. At large lags the proportional controller becomes unstable because it is reacting to positions the sun has already left.

The measured average end-to-end pipeline latency is 2.40 ms. The worst-case measured latency is 4.55 ms. Both are well below the 33 ms. There is no systematic backlog.

A polling approach — where a loop checks whether a new frame is available at regular intervals using `sleep()` — would not work reliably here. Between frames the thread wastes CPU cycles doing nothing useful. When a frame does arrive, the thread might not see it for up to one poll interval. And because `sleep()` timing is subject to OS scheduler decisions, the actual wakeup time is non-deterministic. The event-driven approach used here has none of these problems: the control thread blocks in the kernel and wakes the instant a frame is pushed into its queue.

The same argument applies to the sensors. I2C reads from the ADS1115 and MPU6050 only block for the duration of the transfer itself — they do not wait for new data. If the software polled those devices at a fixed rate, sample timing would depend on the software's scheduling, not the sensor's conversion rate. By using the hardware data-ready interrupt pins (ALERT/RDY and INT), the software sleeps until the sensor itself signals that new data is ready. Sample timing is therefore governed entirely by the hardware.

Stage	Description
Physical event	Sun moves; light pattern on IMX219 sensor shifts.
Sensing	Camera DMA transfer completes. FrameEvent callback fires in camera thread.
Queue handoff	Frame pushed to frame_q_ (capacity 2, push_latest). Control thread woken by condition_variable.
Vision	SunTracker thresholds frame, counts bright pixels, computes weighted centroid. Emits SunEstimate with confidence.
Control	Controller applies proportional gain, deadband, and saturation. Emits PlatformSetpoint in radians.
IMU correction	ManuallmuCoordinator optionally applies tilt offset from latest IMU sample.
Inverse kinematics	Kinematics3RRS: ZYX Euler rotation matrix → cosine-rule per leg → three servo angles in degrees.
Command queue	ActuatorCommand pushed to cmd_q_ (capacity 8). Actuator thread woken by condition_variable.
Safety check	ActuatorManager applies per-channel clamping and slew-rate limiting.
Hardware output	ServoDriver converts degrees to PWM pulse width. Writes to PCA9685 over I2C. PWM pulses drive servos.
Physical response	Servo arms move. Linkage tilts platform. Solar surface faces the sun.

Table 5.1. The complete event-to-actuation chain from light source movement to servo response.

6. Software Architecture

The software is written in C++17 and built with CMake 3.16+ using Ninja. Everything runs inside a single Linux process structured as a forward-only staged pipeline. Each stage owns a specific transformation and communicates with the next stage through a registered callback function — it never queries the upstream stage for data.

6.1 The Automatic Pipeline

```
ICamera → SunTracker → Controller → ManualImuCoordinator → Kinematics3RRS →  
ActuatorManager → ServoDriver
```

Each stage receives typed input and produces typed output. The data types flowing through are:

- `FrameEvent` — raw pixel data, format, dimensions, stride, frame ID, capture timestamp
- `SunEstimate` — centroid (cx, cy) in pixels, confidence in [0,1], frame ID, timestamp
- `PlatformSetpoint` — tilt_rad, pan_rad, frame ID, timestamp
- `ActuatorCommand` — actuator_targets[3] in degrees, status code (Ok / KinematicsFallback / KinematicsInvalidConfig), frame ID

6.2 Class Responsibilities

Class	Responsibility
<code>SystemManager</code>	Thin runtime orchestrator. Owns two worker threads, two bounded queues, and the state machine. About 380 lines.
<code>BackendCoordinator</code>	Owns ADS1115 and MPU6050 backend lifecycle (I2C construction, GPIO setup, start/stop). Exposes only start() and stop() to SystemManager.
<code>GuiManualDispatcher</code>	Dedicated worker thread and capacity-1 bounded queue for GUI manual setpoints. Wakes immediately on operator input and dispatches directly to Kinematics3RRS, independent of camera timing.
<code>ICamera</code>	Pure virtual interface: registerFrameCallback(), start(), stop(), isRunning(). Four methods. Keeps the pipeline independent of which camera backend is in use.
<code>LibcameraPublisher</code>	Concrete ICamera. Delivers frames via libcamera DMA completion callback.
<code>SimulatedPublisher</code>	Concrete ICamera. Uses timerfd + poll() to pace frames at 30 Hz for software testing without hardware.
<code>SunTracker</code>	Vision stage. Thresholds for brightness, counts qualifying pixels, computes weighted centroid. Emits SunEstimate.
<code>Controller</code>	Control stage. Proportional gain with deadband re-scaling and saturation. Emits PlatformSetpoint.
<code>ManualImuCoordinator</code>	Coordinates manual input source ownership and optional IMU tilt correction. Builds manual setpoints for both pot and GUI paths.
<code>Kinematics3RRS</code>	3-RRS inverse kinematics. ZYX Euler rotation matrix + cosine-rule per leg. Branch selection and fallback on unreachable configurations.
<code>ActuatorManager</code>	Safety layer. Per-channel output clamping and optional slew-rate limiting.
<code>ServoDriver</code>	Final output. Converts degrees to PWM pulse widths and writes via PCA9685. Parks servos on start and stop. Three startup policies.
<code>PCA9685</code>	Low-level I2C PWM chip access through the IICDevice interface.

Class	Responsibility
ThreadSafeQueue<T>	Blocking bounded queue. push_latest() drops the oldest item when full. wait_pop() blocks on a condition variable.
LatencyMonitor	Records std::chrono::steady_clock timestamps at four points per frame. Exports CSV and prints summary on shutdown.
LinuxEventLoop	Headless application event loop. Multiplexes signalfd, timerfd, and stdin on a single poll() call.
SystemFactory	Composition root. Selects concrete backends, wires the runtime graph, injects dependencies. The only place that knows about concrete types.

Table 6.1. Class responsibilities. Each class has one reason to change.

6.3 Callback Map

All stage-to-stage communication uses registered std::function callbacks for incoming events, and setter-style methods for outgoing commands. No stage polls an upstream stage.

Publisher	Subscriber	Data Type	Registration
ICamera	SystemManager	FrameEvent	camera_->registerFrameCallback()
SunTracker	SystemManager	SunEstimate	tracker_.registerEstimateCallback()
Controller	SystemManager	PlatformSetpoint	controller_.registerSetpointCallback()
Kinematics3RRS	SystemManager	ActuatorCommand	kinematics_.registerCommandCallback()
ActuatorManager	SystemManager	ActuatorCommand	actuatorMgr_.registerSafeCommandCallback()
IIMU	BackendCoordinator	ImuSample	registerEventCallback()
ADS1115ManualInput	BackendCoordinator	ManualPotSample	registerCallback()
LatencyMonitor	SystemManager	per-frame metrics	latency_.registerObserver()

Table 6.2. Callback registration map.

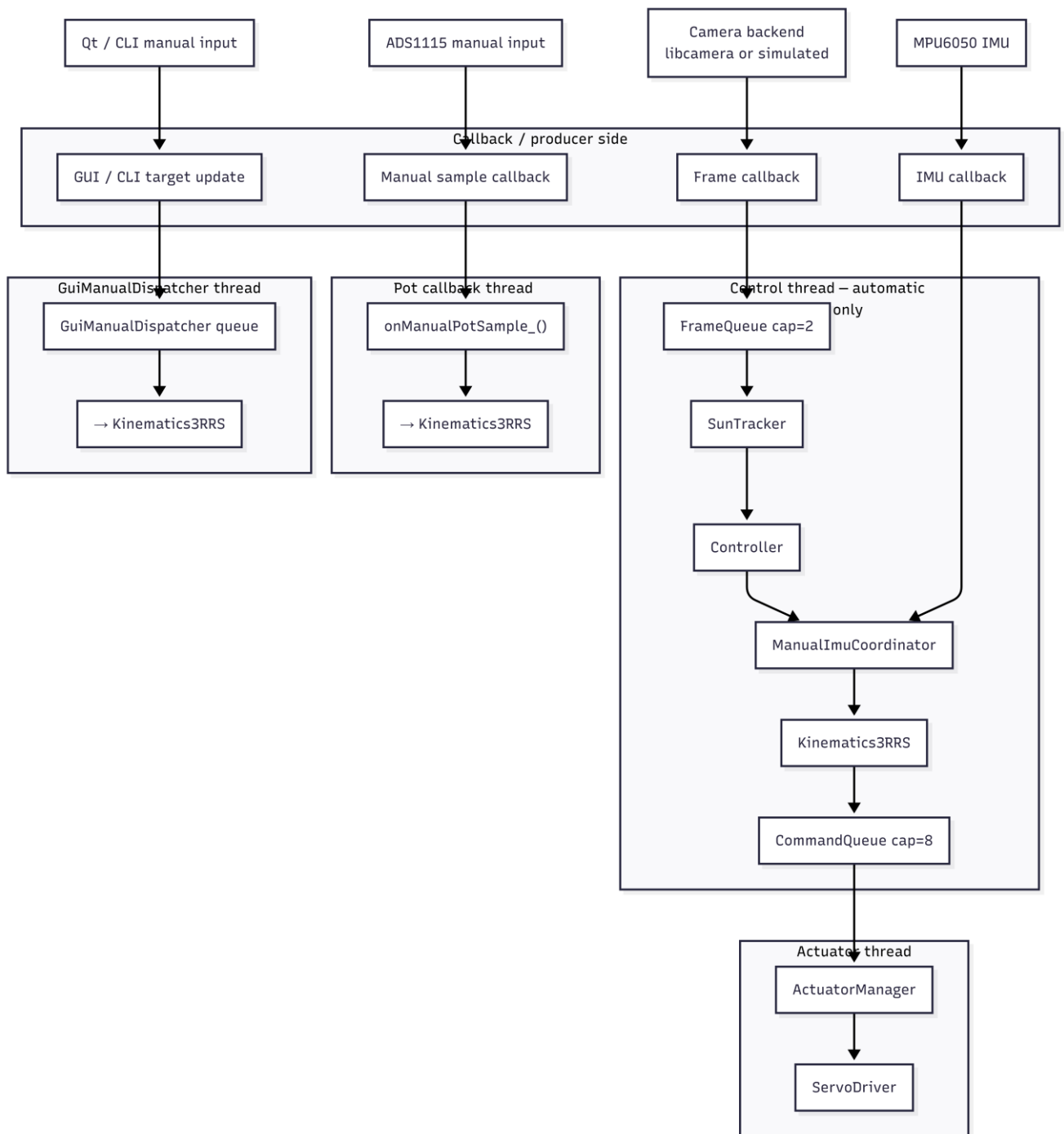


Figure 6.2. Threaded event-driven architecture diagram showing all threads, blocking primitives, queues, and data paths.

7. Realtime and Event-Driven Design

The core design rule throughout the system is that no thread ever checks or polls for work. Every thread blocks on a kernel-backed primitive and wakes only when an external event delivers something to process. This means zero CPU time is spent waiting, the response latency is as low as the kernel scheduler allows, and there is no timing error introduced by fixed-rate polling intervals.

7.1 Camera Path

LibcameraPublisher delivers frames through a libcamera DMA completion callback. When a camera frame is ready, the callback fires in the camera backend thread. The handler records a capture timestamp, pushes the FrameEvent into frame_q_ via push_latest(), and returns immediately — it does nothing slow. The control thread, which has been sleeping on frame_q_.wait_pop(), wakes up and processes the frame.

SimulatedPublisher uses the same downstream path but replaces the DMA callback with a timerfd + poll() loop at 30 Hz. This is the right approach when there is no real hardware event to block on — the timerfd provides a proper kernel-backed event source rather than a sleep() call.

```
// SimulatedPublisher worker thread:
timer_fd_ = timerfd_create(CLOCK_MONOTONIC, TFD_CLOEXEC);
timerfd_settime(timer_fd_, 0, &spec, nullptr); // fires every 33 ms
while (running_) {
    poll(fds, 2, -1); // sleeps until timer fires
    read(timer_fd_, &expirations, sizeof(expirations));
    synthesiseAndDeliverFrame(); // same path as hardware
}
```

7.2 Control Thread

The control thread runs controlLoop_(), which blocks on frame_q_.wait_pop(). When a frame is available, the thread checks the current system state. In SEARCHING or TRACKING it runs the full automatic pipeline: SunTracker → Controller → ManualImuCoordinator → Kinematics3RRS, then pushes the resulting ActuatorCommand into cmd_q_. In MANUAL or FAULT it drains frames without processing to keep the queue clear.

This separation between the fast camera callback (push to queue, return) and the slower vision and control work (done in the control thread) is deliberate. The camera callback must always complete quickly so the libcamera backend is never stalled. Vision processing takes ~2.4 ms on average and has variable runtime — it belongs in a separate blocking thread, not in the callback.

7.3 Actuator Thread

The actuator thread blocks on cmd_q_.wait_pop(). When a command arrives, it passes through ActuatorManager for clamping and slew limiting, then ServoDriver converts it to PWM pulse widths and writes to the PCA9685 over I2C. The thread returns to wait_pop() immediately after.

Separating the actuator thread from the control thread means I2C write timing (0.017 ms average) is decoupled from vision processing jitter (2.38 ms average). Without this separation, servo outputs would be delayed by the full vision processing time on every frame.

7.4 Data-Ready GPIO Pattern — ADS1115 and MPU6050

I2C only blocks for the duration of a register transfer — it does not wait for a sensor to produce new data. Polling I2C at a fixed rate wastes bus bandwidth and ties sample timing to software

scheduling rather than the sensor's conversion rate. The correct approach is to use the sensor's data-ready output pin to signal when a result is available.

Both sensors in this project have such pins:

- ADS1115 ALERT/RDY (GPIO17): fires when a conversion is complete. The ADS1115ManualInput thread blocks on libgpiod wait_edge_events(). When the edge fires, the thread reads the ADC result over I2C and invokes the sample callback. Timing is governed by the ADS1115 conversion rate, not the software.
- MPU6050 INT (GPIO27): fires when a new inertial sample is ready. The Mpu6050Publisher thread blocks on the same libgpiod pattern. When the edge fires, six I2C registers are read and the sample forwarded to ManualImuCoordinator.

```
// Mpu6050Publisher - data-ready GPIO pattern:
gpio_pin_.registerCallback([this](const gpiod::edge_event& event) {
    onGpioEvent_(event);    // only called when INT pin fires
});
gpio_pin_.start(config_.gpio_pin_no, config_.gpio_chip_no);
// The gpio_pin_ thread internally runs:
//   while (running) {
//       request->wait_edge_events(...); // sleeps until interrupt fires
//       read 6 I2C registers             // only then touches the bus
//       invoke callback with sample      // forward to coordinator
//   }
```

7.5 Potentiometer Manual Path

When ADS1115ManualInput fires its callback, the system is in the ADS1115 callback thread. Rather than queuing the sample for the control thread to pick up later, the pot-manual path dispatches directly: ManualImuCoordinator builds a platform setpoint, applies IMU correction if enabled, and calls Kinematics3RRS. This makes manual potentiometer response independent of camera frame timing entirely.

7.6 GUI Manual Path

When the operator moves a slider, the Qt GUI thread calls SystemManager::setManualSetpoint(), which calls GuiManualDispatcher::setSetpoint(). The dispatcher pushes the setpoint to its internal capacity-1 queue and returns. The GuiManualDispatcher worker thread, which has been blocking on that queue, wakes immediately and runs the same dispatch path as the pot: ManualImuCoordinator → Kinematics3RRS → cmd_q_.

Qt timers are used only for display refresh and chart updates. They never trigger actuator commands.

7.7 IMU Path

The MPU6050 callback thread forwards samples to ManualImuCoordinator::updateImuSample(). Samples do not directly trigger actuation. They update the correction offset that gets applied the next time any setpoint passes through applyImuCorrection(). The IMU and the control pipeline are decoupled.

7.8 Application Event Loop

The main thread runs LinuxEventLoop, which multiplexes three file descriptors on a single poll(-1) call. It sleeps until one of three events occurs: a Unix signal (SIGINT/SIGTERM via signalfd, which triggers clean shutdown), a 30 Hz timerfd tick (for CLI heartbeats), or stdin input (for commands). There is no code in main() — it constructs an Application and calls run().

```
// main.cpp - the entire file:
```

```

#include "app/Application.hpp"
int main() {
    solar::app::Application app;
    return app.run();
}

// LinuxEventLoop::run() — all event handling:
int sig_fd = signalfd(-1, &mask, SFD_CLOEXEC); // Unix signals
int timer_fd = timerfd_create(CLOCK_MONOTONIC, ...); // 30 Hz tick
// stdin duplicated as fds[2]
while (!done) {
    poll(fds, 3, -1); // sleeps until any event
    if (fds[0].revents & POLLIN) handleSignal(); // SIGINT → shutdown
    if (fds[1].revents & POLLIN) cli_.onTick(); // CLI heartbeat
    if (fds[2].revents & POLLIN) cli_.onInput(); // user command
}

```

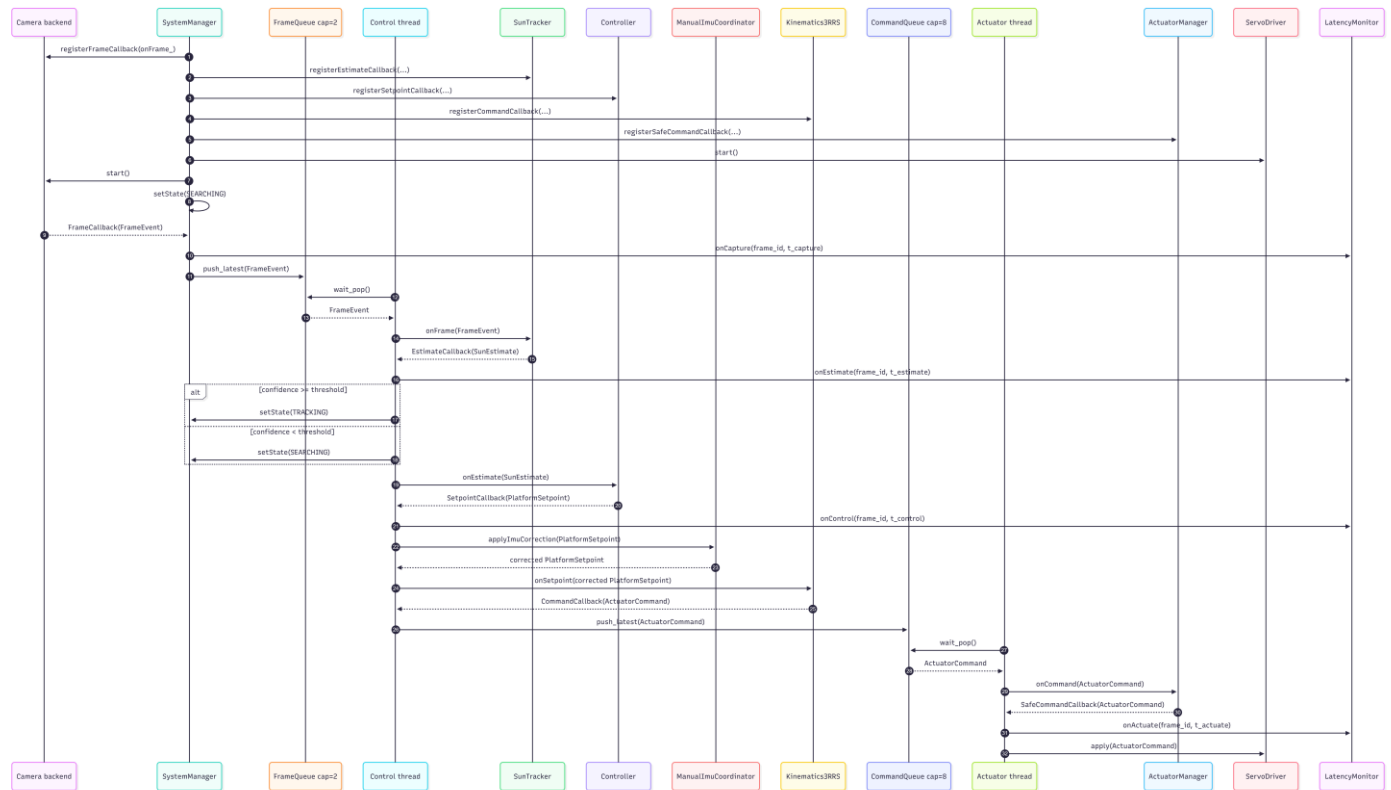


Figure 7.1. Sequence diagram — automatic pipeline from start() to steady-state tracking. Every inter-stage transition is callback-driven or queue-pushed.

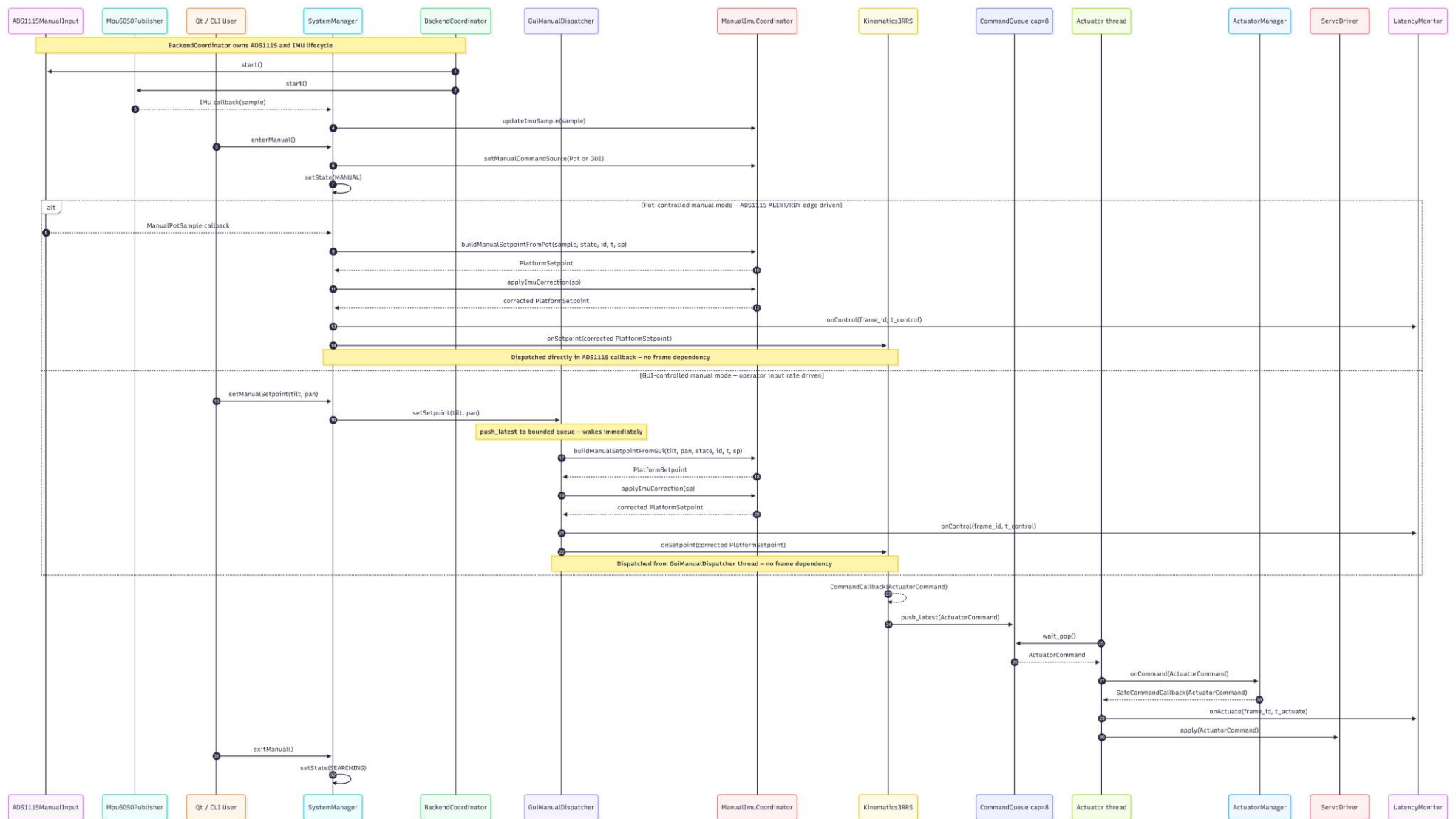


Figure 7.2. Sequence diagram — manual input and IMU update paths. Both pot and GUI paths operate independently of camera frames.

8. Threading Model

The system uses eight threads. Each one blocks on a kernel-backed primitive and has a single, named role. The table below lists them all.

Thread	Blocking Primitive	Work Performed	Output
Camera (hardware)	libcamera DMA callback	Receive FrameEvent from libcamera	frame_q_.push_latest()
Camera (simulated)	timerfd + poll()	Synthesise frame at 30 Hz	frame_q_.push_latest()
Control	frame_q_.wait_pop()	SunTracker → Controller → ManualImuCoordinator → Kinematics3RRS	cmd_q_.push_latest()
Actuator	cmd_q_.wait_pop()	ActuatorManager conditioning	ServoDriver.apply() → I2C
GuiManualDispatcher	queue condition_variable	Build manual setpoint via ManualImuCoordinator	Kinematics3RRS → cmd_q_
ADS1115	GPIO ALERT/RDY (libgpiod)	Read ADC result over I2C	ManualImuCoordinator → Kinematics3RRS → cmd_q_
MPU6050	GPIO INT (libgpiod)	Read 6 IMU registers over I2C	ManualImuCoordinator.updateImuSample()
Main / event loop	poll() on signalfd/timerfd/stdin	Signal handling, CLI commands	SystemManager.start/stop/enterManual

Table 8.1. All eight threads with blocking primitive, work, and output. No thread uses *sleep()*, *polling*, or *busy-waiting*.

The frame queue (capacity 2) separates the camera callback thread from the control thread. The command queue (capacity 8) separates the control thread from the actuator thread. Both queues use the *push_latest()* policy: when a queue is full, the oldest item is dropped before the newest is inserted. This keeps both queues bounded in size and ensures the consumer always gets the most recent data.

All three processing paths — automatic, pot-manual, and GUI-manual — converge at Kinematics3RRS. Because all three can call Kinematics3RRS concurrently, the IK solver protects its internal state (*q_prev_* and *last_valid_deg_*) with a mutex (*ik_mtx_*).

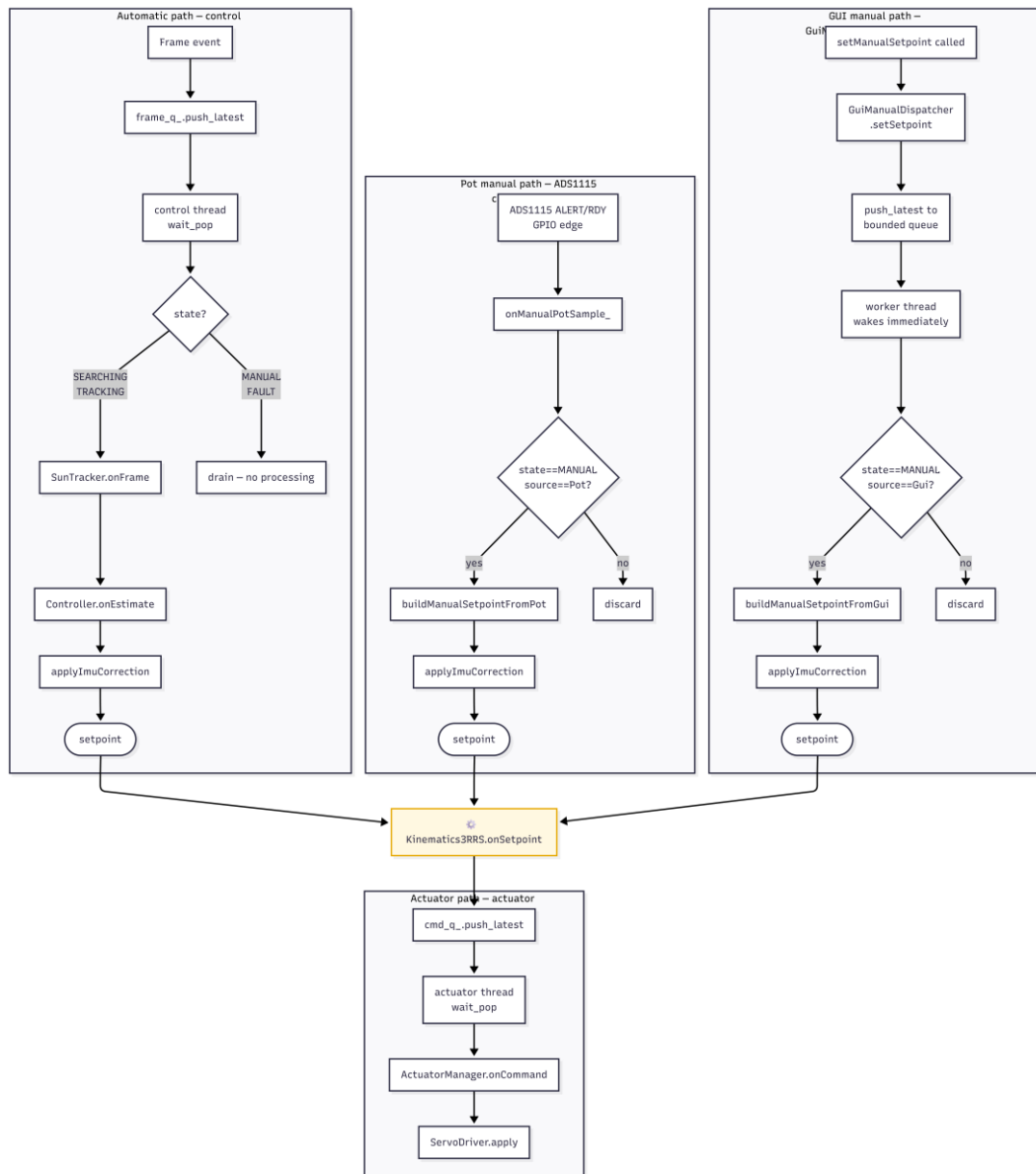


Figure 8.1. Automatic vs manual processing paths — thread boundaries, state-conditional branching in the control thread, and convergence at Kinematics3RRS.

9. Dataflow and Control Pipelines

9.1 Automatic Path (control thread)

```
Camera DMA callback
→ LatencyMonitor.onCapture(frame_id, t_capture)
→ frame_q_.push_latest(FrameEvent)

[control thread wakes from wait_pop()]
→ SunTracker.onFrame(FrameEvent)
  estimate_cb_(SunEstimate)
→ LatencyMonitor.onEstimate(frame_id, t_estimate)
→ setState(TRACKING if confidence >= threshold, else SEARCHING)
→ Controller.onEstimate(SunEstimate)
  setpoint_cb_(PlatformSetpoint)
→ LatencyMonitor.onControl(frame_id, t_control)
→ ManualImuCoordinator.applyImuCorrection(setpoint)
→ Kinematics3RRS.onSetpoint(corrected setpoint)
  command_cb_(ActuatorCommand)
→ cmd_q_.push_latest(ActuatorCommand)

[actuator thread wakes from wait_pop()]
→ ActuatorManager.onCommand(ActuatorCommand)
  safe_command_cb_(safe ActuatorCommand)
→ LatencyMonitor.onActuate(frame_id, t_actuate)
→ ServoDriver.apply(command) → PCA9685 I2C write → PWM → Servos
```

9.2 Potentiometer Manual Path (ADS1115 callback thread)

```
ADS1115 ALERT/RDY GPIO edge (libgpiod, GPIO17)
→ onManualPotSample_() → ManualImuCoordinator.buildManualSetpointFromPot()
→ applyImuCorrection() → Kinematics3RRS → cmd_q_ → actuator thread →
ServoDriver
```

9.3 GUI Manual Path (GuiManualDispatcher thread)

```
Qt slider moved → SystemManager.setManualSetpoint(tilt, pan)
→ GuiManualDispatcher.setSetpoint() → queue.push_latest()

[GuiManualDispatcher thread wakes immediately]
→ ManualImuCoordinator.buildManualSetpointFromGui()
→ applyImuCorrection() → Kinematics3RRS → cmd_q_ → actuator thread →
ServoDriver
```

9.4 IMU Update Path (MPU6050 callback thread)

```
MPU6050 INT GPIO edge (libgpiod, GPIO27)
→ Mpu6050Publisher reads 6 I2C registers
→ ManualImuCoordinator.updateImuSample()
[updates correction offset – no actuation triggered]
```

10. Software Engineering and Class Design

The class structure follows the SOLID principles — not as a checklist, but because they solve real problems in a system that combines hardware access, realtime processing, and testability.

10.1 Single Responsibility

Each class has one reason to change. SunTracker changes if the detection algorithm changes — it has no knowledge of servos. Controller changes if the control law changes — it has no knowledge of the Stewart mechanism geometry. Kinematics3RRS changes if the mechanism geometry changes — it has no knowledge of cameras. ServoDriver changes if PWM calibration or PCA9685 access changes — it does not implement control policy. SystemManager changes if orchestration or state machine logic changes — it does not implement vision, kinematics, or hardware access, and sits at around 380 lines.

10.2 Open/Closed

ICamera allows new camera backends to be added without modifying SystemManager or any downstream stage. LibcameraPublisher and SimulatedPublisher are complete, independent implementations selected at construction time in SystemFactory. ServoDriver's three startup policies (LogOnly, PreferHardware, RequireHardware) change behaviour without subclassing or branching in calling code.

10.3 Interface Segregation

ICamera exposes only four methods. SystemManager depends on this narrow interface, not on libcamera or OpenCV. BackendCoordinator exposes only start() and stop() to SystemManager — all I2C device construction, GPIO chip setup, and interrupt routing are internal. GuiManualDispatcher exposes only setSetpoint(), start(), and stop() — its queue and worker thread are private.

10.4 Dependency Inversion

The hardware abstraction layer defines the II2CDevice interface. PCA9685 depends on II2CDevice, not on LinuxI2CDevice. In tests, FakeI2CDevice is injected — PCA9685 register sequences can be verified without physical hardware. SystemManager depends on ICamera (not LibcameraPublisher), ManualMuCoordinator (not ADS1115 hardware directly), and GuiManualDispatcher (not Qt directly). SystemFactory is the only place that knows about concrete types.

10.5 Data Encapsulation

All mutable state is private. There are no public member variables anywhere. Incoming events are delivered through registered callbacks — nothing is exposed for external polling. Outgoing control commands use setter-style methods (onEstimate(), onSetpoint(), onCommand(), apply()). Every field accessed across threads is protected by a dedicated std::mutex, with short critical sections and copy-under-lock patterns.

10.6 Memory Safety

The camera backend is held as std::unique_ptr<ICamera>. Callbacks are stored as std::function. Threads are std::thread with RAII join in stop() or destructors. ThreadSafeQueue uses std::deque<T> with an explicit capacity bound. There is no raw owning pointer, no new/delete, and no malloc/free in user code.



11. Key Source Code

This section annotates the most important code passages to show that the design described above is actually implemented that way.

11.1 ThreadSafeQueue — push_latest() and wait_pop()

These two methods are the core of the inter-thread handoff. `push_latest()` enforces the freshest-data policy. `wait_pop()` is where threads spend most of their time — sleeping.

```
bool push_latest(T item) {
    {
        std::lock_guard<std::mutex> lock(mutex_);
        if (stopped_) { return false; }
        // At capacity: drop oldest, insert newest.
        if (capacity_ > 0U && queue_.size() >= capacity_) {
            queue_.pop_front();
        }
        queue_.push_back(std::move(item));
    }
    cv_.notify_one();    // wake exactly one sleeping consumer
    return true;
}

std::optional<T> wait_pop() {
    std::unique_lock<std::mutex> lock(mutex_);
    cv_.wait(lock, [this] { return stopped_ || !queue_.empty(); });
    if (queue_.empty()) return std::nullopt;    // woken by stop()
    T item = std::move(queue_.front());
    queue_.pop_front();
    return item;
}
```

The queue never grows beyond capacity. Under transient load, stale data is discarded rather than allowed to build up. `stop()` wakes all blocked threads cleanly so they can exit.

11.2 SunTracker — Weighted Centroid

The tracker scans the frame once, accumulates brightness-weighted pixel coordinates, and computes the centroid. Confidence grows with the number of pixels above threshold.

```
for (int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
        const uint8_t px = intensityAt_(frame, x, y);
        if (px < threshold) continue;
        ++bright_count;
        const double w = static_cast<double>(px);
        weight_sum += w;
        weighted_sum_x += w * x;
        weighted_sum_y += w * y;
    }
}
estimate.cx = weighted_sum_x / weight_sum;
estimate.cy = weighted_sum_y / weight_sum;

// Confidence rises from 0 at min_pixels to 1 at 10 * min_pixels.
double ratio = bright_count / (double)min_pixels;
estimate.confidence = clamp((ratio - 1.0) / 9.0, 0.0, 1.0);
```

RGB and BGR frames are converted to luminance using integer arithmetic: $(77R + 150G + 29B) >> 8$. This avoids floating-point per-pixel cost. Frame validity (dimensions, stride, buffer size) is checked once before the loop.

11.3 Controller — Deadband with Range Re-Scaling

```
// Normalise pixel error to [-1, +1] relative to image centre.
float ex = (est.cx - cx) / cx;
float ey = (est.cy - cy) / cy;

// Deadband: ignore small errors to prevent jitter at null.
// Re-scale the remaining range so full output authority is preserved.
float applyDeadband_(float v) {
    if (fabs(v) < deadband) return 0.0f;
    float sign = v > 0 ? 1.0f : -1.0f;
    return sign * (fabs(v) - deadband) / (1.0f - deadband);
}

float pan = clamp(k_pan * applyDeadband_(ex), -max_pan, max_pan);
float tilt = clamp(k_tilt * applyDeadband_(ey), -max_tilt, max_tilt);
```

The re-scaling step after the deadband is important. Without it, the output would jump discontinuously from 0 at the deadband edge to a small non-zero value just outside it. The re-scaling makes the transition smooth while still zeroing small errors.

11.4 Kinematics3RRS — Inverse Kinematics Core

```
// ZYX intrinsic Euler rotation matrix from (tilt_rad, pan_rad).
R = { cp,    spc*sr,  spc*cr,
      0,     cr,     -sr,
      -spc,  cp*sr,  cp*cr };

for each leg i:
    // Rotate platform anchor into global frame, add home height.
    Px = R[0][0]*Px_l + R[0][1]*Py_l;
    Pz = R[2][0]*Px_l + R[2][1]*Py_l + h;

    // Cosine rule gives the horn angle.
    C = (dx2 + dy2 + dz2 + L12 - L22) / (2 * L1);
    ratio = C / hypot(x, z);

    if (ratio < -1 || ratio > 1) {
        // Configuration unreachable — hold last valid angle.
        cmd.actuator_targets[i] = last_valid_deg[i];
        usedFallback = true; continue;
    }

    // Two geometric solutions; prefer the one closest to previous angle.
    q = chooseClosest(gamma + acos(ratio), gamma - acos(ratio), q_prev[i]);
    // Also prefer horn-up (cos(q) >= 0) to avoid singularities.
    if (cos(q) < 0) q = otherBranch(qA, qB, q);

    servo_deg = clamp(neutral_deg[i] + dir[i] * rad2deg(q), 0, 180);
```

Branch selection prevents discontinuous jumps between geometrically equivalent solutions as the platform moves. The `ik_mtx_mutex` protects `q_prev` and `last_valid_deg` against concurrent calls from the three dispatch paths.

11.5 ServoDriver — Startup Policies

```

switch (cfg_.startup_policy) {
    case LogOnly:
        hardware_mode_ = false;           // never try hardware – for CI
        break;
    case PreferHardware:
        // hardware_mode_ already reflects PCA9685.start() result
        break;
    case RequireHardware:
        if (!hardware_mode_) {
            log_.error("PCA9685 required but failed to start");
            return false;                 // propagates to FAULT state
        }
        break;
}
if (cfg_.park_on_start) applyPark_();    // safe position on every
startup

```

Three policies in one class cover three use cases: software testing without hardware, field deployment with graceful degradation, and production hardware with explicit failure. The policy selection avoids subclassing and keeps all servo output behaviour in one place.

12. Testing and Reliability

Tests sit alongside their source under `src/*/tests/` and are integrated with CTest. A small custom framework (`src/tests/support/`) avoids an external dependency. Two tiers exist: 55 software tests that run without hardware, and 20 hardware-adjacent tests enabled with `-DSOLAR_ENABLE_HW_TESTS=ON`.

12.1 Software Test Matrix

Module Under Test	Cases Covered
SunTracker (3)	Bright pixel detection; zero confidence when no bright region; weighted centroid correctness on synthetic input.
Controller (4)	Confidence gating — no motion below threshold; deadband suppression; non-zero response outside deadband; output saturation.
Kinematics3RRS (9)	Output range for moderate setpoints; neutral symmetry; frame ID propagation; continuity for small changes; invalid geometry error surfacing; fallback to last valid; consecutive invalid configs; branch stability.
ManualInputMapper (5)	Centre-voltage zero mapping; full-range linear mapping; inversion flags; deadband re-scaling; out-of-range clamping.
ImuTiltEstimator (5)	Invalid sample handling; level orientation; positive/negative tilt reconstruction; degenerate denominator protection.
ImuFeedbackMapper (4)	Zero-tilt zero-correction; sign correctness; symmetry; runtime gain change takes effect immediately.
ActuatorManager (5)	First-command behaviour; per-channel rate limiting; saturation ordering relative to history update; history reset; zero-step hold.
ThreadSafeQueue (9)	FIFO ordering; strict bounded rejection; <code>push_latest</code> drop-oldest; blocking wake; stop/unblock; concurrent producer-consumer integrity; freshest-data under contention; stress test.
LatencyMonitor (3)	Ordered timestamp handling; out-of-order robustness; in-flight pruning under pressure.
SystemManager (15)	IDLE→STARTUP→SEARCHING→TRACKING progression; manual mode command emission; null-camera FAULT; camera-start FAULT; IMU shadow/disabled/live variants; state observer transitions; end-to-end pipeline; sustained multi-frame throughput; GUI manual path independence from camera frames.

Table 12.1. Software test matrix — 55 CTest entries, all runnable without hardware.

12.2 Hardware-Adjacent Tests

Target	Cases
PCA9685 (4)	Safe default construction; pre-start failure; channel validation; safe stop before startup.
ServoDriver (9)	LogOnly/PreferHardware/RequireHardware startup; injected PCA path; no writes while stopped; park on start/stop; channel clamping; inverted channel mapping.
Mpu6050Publisher (6)	Callback pre-condition; WHO_AM_I identity check; six init register writes; full start/stop lifecycle; double-start guard; startup discard window.
LinuxI2CDevice (1)	I2C hardware smoke path. Skips cleanly (SKIP_RETURN_CODE 77) when hardware is absent.

Table 12.2. Hardware-adjacent tests — 20 additional CTest entries enabled with `-DSOLAR_ENABLE_HW_TESTS=ON`.

```
# Run software tests (no hardware needed)
ctest --test-dir build --output-on-failure

# Run hardware-adjacent tests on Raspberry Pi
SOLAR_RUN_I2C_HW_TESTS=1 ctest --test-dir build -L hw --output-on-failure
```

12.3 Fault Handling

FAULT state is entered on: null camera at startup; ServoDriver RequireHardware failure; BackendCoordinator failure; camera stream failure; invalid kinematics result at runtime. In FAULT, actuator commands are suppressed. stop() is safe from FAULT and proceeds to STOPPING → IDLE.

On both startup and shutdown, ServoDriver moves all servos to a safe park position (configurable via park_on_start and park_on_stop). This prevents the platform from being in an unknown pose when the process starts or exits.

12.4 Shutdown

stop() is safe from any state and can be called multiple times. The STOPPING sequence is ordered: camera.stop() → backends.stop() → GuiManualDispatcher.stop() → frame_q_.stop() and cmd_q_.stop() (notify_all() wakes any blocked threads) → control thread join → actuator thread join → neutral servo command → driver.stop() → state = IDLE.

13. Realtime Performance and Latency

Latency was measured on real Raspberry Pi 5 hardware with physical servos connected and moving. The goal was to get actual numbers that could inform specific architectural decisions, not just confirm the system works.

13.1 Measurement Platform

Parameter	Value
Hardware	Raspberry Pi 5, BCM2712, Cortex-A76 quad-core, 4 GB RAM
OS	Raspberry Pi OS (Debian Trixie), 64-bit (aarch64), kernel 6.6.51+rpt
Build	GCC 12.2.0, Release (-O2)
Camera backend	libcamera2opencv, IMX219 CSI, 640×480, RGB888, 30 fps
Servo backend	RequireHardware — physical servos connected and moving throughout
Dataset	1367 frames over 46.6 seconds
Tool	LatencyMonitor using std::chrono::steady_clock per frame

Table 13.1. Measurement platform.

13.2 Results

Metric	Average (ms)	Min (ms)	Max (ms)	Jitter (ms)
L_total (end-to-end)	2.398	1.870	4.553	2.683
L_vision (capture → estimate)	2.379	1.860	4.531	2.672
L_control (estimate → control)	0.0015	0.0005	0.0703	0.0698
L_actuation (control → actuate)	0.0172	0.0086	0.1563	0.1477

Table 13.2. Measured pipeline latency over 1367 frames on Raspberry Pi 5. Raw data: [artefacts/latency.csv](#).

13.3 Interpretation

Vision processing (SunTracker) dominates, accounting for 2.38 ms — 99.2% of total latency. The control and kinematics stages add only 0.0015 ms. Actuator I2C writes take 0.017 ms on average.

Average end-to-end latency of 2.40 ms is 7.3% of the 33 ms frame period at 30 fps. Worst-case 4.55 ms is 13.8%. The system never comes close to missing a frame deadline.

The frame-interval standard deviation of 4.46 ms confirms these measurements came from real camera hardware. A timerfd-driven simulated camera produces sub-millisecond interval variation. The 4.46 ms spread reflects libcamera DMA timing variation and Linux scheduler effects — genuine hardware behaviour.

13.4 Design Decisions Justified by These Numbers

Decision	Why the data supports it
Frame queue capacity = 2	Average latency (2.40 ms) is well below one frame period (33 ms). A capacity of 2 absorbs a single-frame control-thread stall without blocking the camera callback. A larger capacity would allow stale frames to accumulate.

Decision	Why the data supports it
Command queue capacity = 8	The actuator stage (0.017 ms) is about 140× faster than vision (2.38 ms). A larger command queue absorbs short bursts from kinematic output without dropping commands prematurely.
30 Hz camera frame rate	Worst-case latency (4.55 ms) is 13.8% of the frame period. A faster rate would not reduce tracking latency because vision dominates. A slower rate would reduce responsiveness.
Separate control and actuator threads	Vision jitter (average 2.38 ms, worst-case 4.55 ms) is isolated from servo writes (0.017 ms). Without this separation, servo timing would vary by the full vision processing time on every frame.
push_latest() freshest-data policy	Under transient load the queue bounds memory use and discards stale data. Without it, frames would backlog and the control loop would be tracking a past position.

Table 13.3. Five architectural decisions directly justified by the measured latency data.

13.5 How Latency Is Measured

```
// LatencyMonitor records four timestamps per frame:
t_capture  - frame received by onFrame_() callback
t_estimate - SunTracker estimate callback fires
t_control  - control / IMU correction complete
t_actuate  - ActuatorCommand pushed to cmd_q_

// Latency definitions:
L_vision   = t_estimate - t_capture
L_control  = t_control  - t_estimate
L_actuation = t_actuate - t_control
L_total    = t_actuate - t_capture

// To capture a CSV on shutdown:
SOLAR_LATENCY_CSV=artefacts/latency.csv ./build/solar_tracker
```

13.6 Scope of Measurements

These numbers cover the userspace software pipeline only. Not included: camera sensor exposure time, libcamera kernel DMA buffering before userspace delivery, PCA9685 PWM propagation, servo electrical response time, or mechanical settling. The system runs as a standard Linux CFS-scheduled process, not with SCHED_FIFO priority or a PREEMPT_RT kernel.

14. Build and Reproducibility

Target platform: Raspberry Pi OS (Debian Trixie), 64-bit (aarch64), kernel 6.6+. libgpiod v2 must be built from source because the apt package provides v1.x only, which lacks the C++ bindings required.

14.1 Dependencies

```
# Core build tools
sudo apt install -y build-essential cmake ninja-build git pkg-config

# libgpiod v2 - build from source
sudo apt install -y autoconf autoconf-archive automake libtool wget xz-utils
cd /tmp && wget
https://mirrors.edge.kernel.org/pub/software/libs/libgpiod/libgpiod-2.2.2.tar.xz
tar -xf libgpiod-2.2.2.tar.xz && cd libgpiod-2.2.2
./configure --prefix=/usr/local --enable-tools --enable-bindings-cxx
make -j$(nproc) && sudo make install && sudo ldconfig

# Optional: hardware camera, Qt GUI, documentation
sudo apt install -y libopencv-dev libcamera-dev
sudo apt install -y qtbase5-dev libqt5charts5-dev
sudo apt install -y doxygen graphviz
```

14.2 Clone and Build

```
# Clone including submodules - do not use the GitHub ZIP
git clone --recurse-submodules \
  https://github.com/Real-Time-Stewart-Solar-Tracker/Solar-Stewart-Tracker.git
cd Solar-Stewart-Tracker

# Full release build (CLI + Qt GUI + all tests)
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build -j

# Minimal headless build (no Qt, no libcamera)
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release \
  -DSOLAR_ENABLE_QT=OFF -DSOLAR_TRY_LIBCAMERA=OFF -DSOLAR_TRY_OPENCV=OFF
cmake --build build -j
```

14.3 Run Commands

Command	Description
<code>./build/solar_tracker</code>	Headless CLI. Commands: manual / auto / set / threshold / quit / help.
<code>./build/src/qt/solar_tracker_qt</code>	Qt GUI with live camera preview, actuation charts, manual sliders.
<code>ctest --test-dir build --output-on-failure</code>	Run all 55 software CTest entries.
<code>./scripts/test_pi_hw.sh</code>	Run hardware-adjacent tests on connected Raspberry Pi.
<code>SOLAR_LATENCY_CSV=artefacts/latency.csv</code> <code>./build/solar_tracker</code>	Run and export latency CSV on shutdown.
<code>doxygen Doxyfile</code>	Generate API documentation to docs/html/.

Table 14.1. Run commands. Hardware mode requires I2C devices connected and visible on `/dev/i2c-1`.

14.4 Repository Structure

```
src/
  actuators/      PCA9685, ServoDriver, ActuatorManager + tests/
  app/            Application, LinuxEventLoop, CliController, SystemFactory,
AppConfig
  common/        ThreadSafeQueue, LatencyMonitor, Logger, Types
  control/       Controller, Kinematics3RRS, ManualImuCoordinator,
ManualInputMapper, ImuTiltEstimator, ImuFeedbackMapper + tests/
  hal/           II2CDevice, LinuxI2CDevice + tests/
  qt/            MainWindow, Qt entry point
  sensors/       LibcameraPublisher, SimulatedPublisher,
ADS1115ManualInput, Mpu6050Publisher + tests/
  system/        SystemManager, BackendCoordinator, GuiManualDispatcher + tests/
  vision/        SunTracker + tests/
diagrams/        Circuit diagram, UML, architecture, state machine diagrams
  mechanical model/ SolidWorks 3-RRS CAD assembly
docs/            system_architecture, realtime_analysis, latency_measurement,
testing, solid_justification, requirements, BOM,
hardware_connections, build_and_run, REPRODUCIBILITY
artefacts/       latency.csv – measured on Raspberry Pi 5 hardware
external/        libcamera2opencv, libgpiod_event_demo (git submodules)
scripts/         build_core, build_pi_debug, test_core, test_pi_hw,
run_latency, build_docs, package_release
.github/workflows/ ci.yml, doxygen.yml, pi-hardware-tests.yml, release.yml
```

15. Project Management and Revision Control

15.1 Team Responsibilities

Name	ID	Area of Responsibility
Fadi Halteh	3127931H	System architecture design. Runtime orchestration (SystemManager, BackendCoordinator, GuiManualDispatcher). IMU and ADS1115 event-driven integration. State machine design. All technical architecture diagrams.
Jichao Wang	3137140W	3-RRS inverse kinematics solver (Kinematics3RRS). Application factory (SystemFactory). Configuration (AppConfig). Entry point structure.
Ziming Yan	2429452Y	Vision subsystem (SunTracker). Qt GUI (MainWindow, live camera preview, per-channel charts, manual sliders).
Tareq A M Almzanin	3139787A	Control layer (Controller closed-loop logic, proportional control, deadband, manual override). Shared data type definitions.
Zhenyu Zhu	3099498Z	Low-level actuator interface (PCA9685, ServoDriver). Latency instrumentation (LatencyMonitor). Timing analysis and CSV export.

15.2 Development Workflow

Development used GitHub Milestones to track major deliverables, Issues for individual tasks and bugs, Pull Requests for code review and integration, and tagged Releases for stable snapshots. Branches followed a feature/<topic> or fix/<topic> naming convention.

Every pull request required: a description of what changed and why, a note on how it was tested, and confirmation that the realtime path introduces no polling loops or sleep-based timing.

15.3 CI/CD

Workflow	Action
ci.yml	Every push: cmake Release build + 55 software CTest entries. Status badge in README.
doxygen.yml	Merge to main: generate Doxygen HTML and deploy to GitHub Pages.
pi-hardware-tests.yml	Tagged push: build and run hardware-adjacent tests on self-hosted Raspberry Pi runner.
release.yml	Tag push: package binary, docs, and latency CSV as a release artefact.

Table 15.1. GitHub Actions CI/CD workflows.

16. Limitations and Future Improvements

Area	Current Limitation
Latency scope	Measurements cover userspace software only. Camera exposure time, kernel DMA latency, servo electrical response, and mechanical settling are not measured. Angular tracking error against a reference was not quantified.
Vision robustness	SunTracker uses brightness thresholding only. In partially cloudy or high-glare conditions, multiple bright patches produce erratic centroid estimates. There is no temporal filtering or multi-region selection.
IMU live feedback	The IMU live-correction loop was not characterised for stability margins. The default deployment uses Shadow mode, which logs IMU data but does not apply correction. Combined gain interactions between the proportional controller and the IMU correction are not analytically bounded.
OS scheduling	The system runs as a standard Linux CFS-scheduled process. No SCHED_FIFO priority or PREEMPT_RT kernel was applied. Worst-case wakeup latency from other processes on the system is not bounded in software.
Kinematic calibration	Geometric parameters are set manually. Physical assembly tolerances introduce a static positioning error that is not corrected at runtime.
Energy measurement	No solar panel or power measurement circuit was included. The energy-yield improvement over a fixed panel was not quantified.

Table 16.1. Known limitations.

16.2 Recommended Next Steps

- Apply SCHED_FIFO scheduling to the control and actuator threads via `sched_setscheduler()` to bound worst-case wakeup latency under system load.
- Add temporal filtering — for example, an exponential moving average on the centroid — and multi-region selection in SunTracker to handle partial cloud cover.
- Characterise the IMU live-correction loop for stability before enabling it by default. Compute the combined loop gain and verify stability margins.
- Add per-servo position encoders or hall-effect sensors to close the kinematic calibration loop at runtime.
- Include a pyranometer and a panel current/voltage measurement to quantify energy-yield improvement against a fixed-panel baseline.
- Extend latency measurement to include PCA9685 I2C write duration and servo step response for a complete end-to-end timing picture.

17. Conclusion

The Solar Stewart Tracker is a working realtime embedded system that continuously aims a 3-RRS Stewart platform toward a bright light source using event-driven camera-based tracking on a Raspberry Pi 5. It is implemented in C++17 as a single Linux userspace process combining libcamera frame acquisition, brightness-thresholding vision, proportional closed-loop control, 3-RRS inverse kinematics, and PCA9685-driven servo actuation.

The realtime architecture works because every thread in the system blocks on a kernel-backed primitive and wakes only when there is actual work to do. Camera frames arrive via libcamera DMA callback. Sensor samples arrive via hardware GPIO interrupts (ADS1115 ALERT/RDY on GPIO17, MPU6050 INT on GPIO27). The application event loop sleeps on `poll()`. No thread polls, no thread sleeps on a timer in the control path, and no thread busy-waits. The camera callback does minimal work (push to queue and return) so it never stalls the camera backend. The slower vision processing runs in a separate blocking thread.

Inter-class communication uses registered `std::function` callbacks for incoming events and setter-style methods for outgoing commands. The pipeline is forward-only — no stage queries an upstream stage. `main()` is four lines.

Latency measurements on real Raspberry Pi 5 hardware over 1367 frames show average end-to-end software latency of 2.40 ms and worst-case of 4.55 ms, against a 33 ms frame period budget. These measurements directly informed five specific architectural decisions: the queue capacities, the frame rate, the thread separation between control and actuation, and the freshest-data queue policy.

The class structure separates vision, control, kinematics, safety conditioning, hardware output, and orchestration into classes with single, focused responsibilities. 55 software unit tests and 20 hardware-adjacent tests run in CI on every commit. Doxygen documentation is generated and published automatically. The main limitation is that physical tracking accuracy — angular error, servo settling time, and energy yield — was not measured; software-side latency is bounded, but the full physical loop remains uncharacterised.

Appendix A. Runtime State Machine

State is held as `std::atomic<TrackerState>` for lock-free reads from any thread. All transitions call `setState_()` which logs the change. FAULT is reachable from any active state. STOPPING is reachable from every state including FAULT. STOPPING always ends in IDLE.

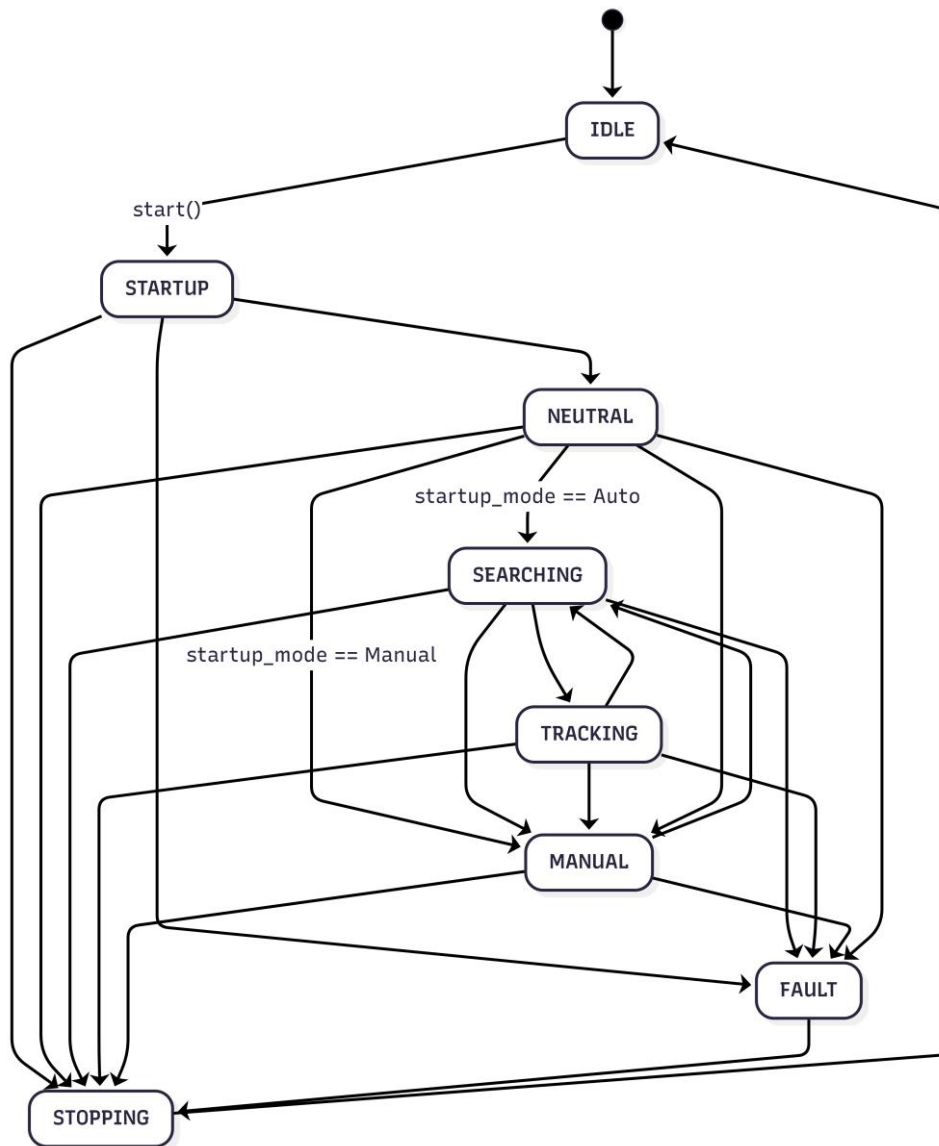


Figure A.1. Full runtime state graph.

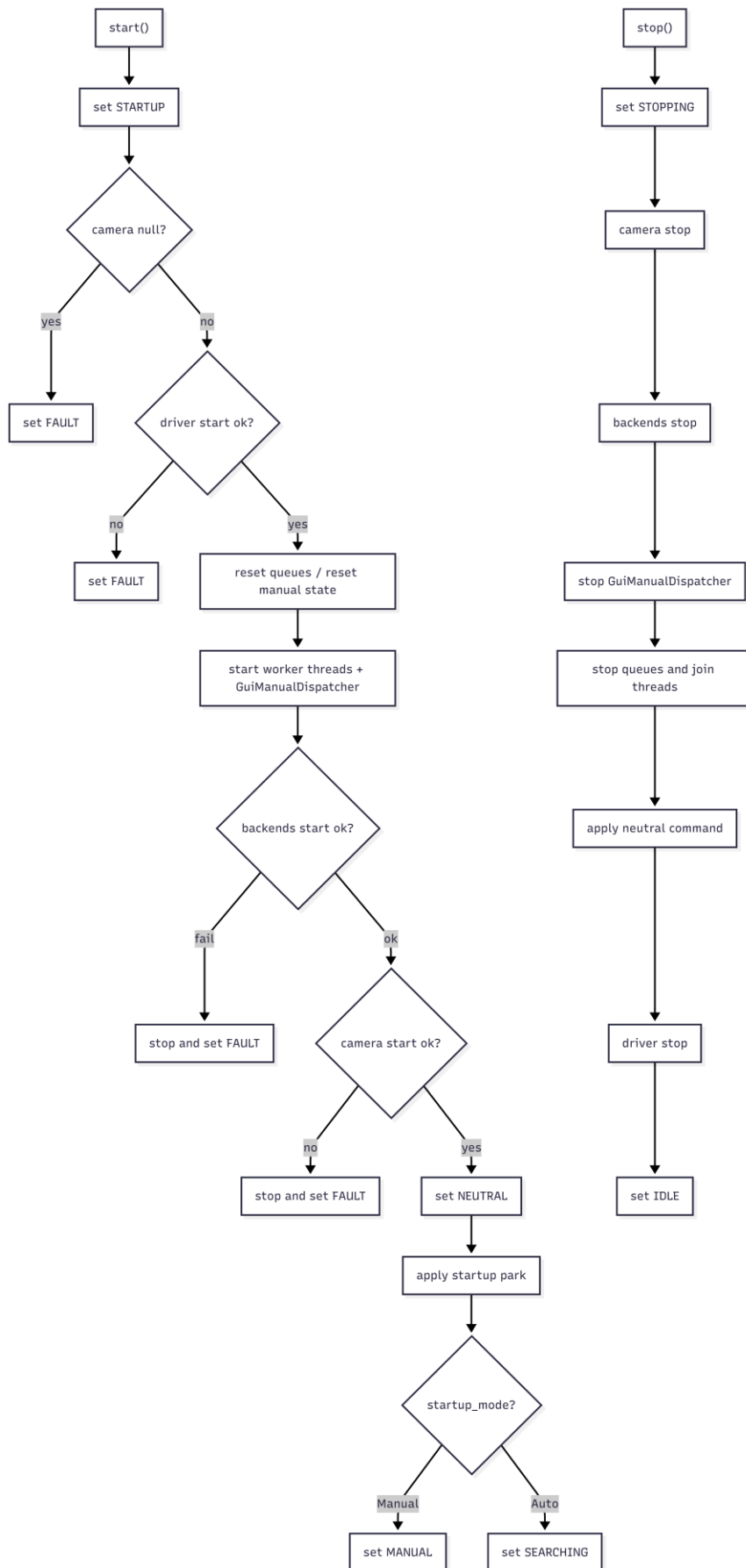


Figure A.2. Startup and shutdown flowchart — decision points, fault conditions, and the ordered cleanup sequence.